Neha Deshmukh

neha.deshmukh@example.com | +91 98997 48105 | https://github.com/nehadeshmukh56 | https://linkedin.com/in/nehadeshmukh56 | https://nehadeshmukh56.dev

EDUCATION

Master of Technology in Software Systems

Indian Institute of Technology Bombay
CGPA: 9.10/10.0 (2020-2024)

SKILLS

Languages
Go, Java, Python, TypeScript, C++, SQL

Frameworks & Systems
Spring Boot, Node.js, FastAPI, React, Next.js, gRPC

Databases & Storage
PostgreSQL, Redis, MongoDB, Apache Kafka, Elasticsearch

Cloud & DevOps
Docker, Kubernetes, AWS (EC2, S3, RDS), Git, CI/CD Pipelines

EXPERIENCE

Associate Software Engineer — Groww (Nextbillion.ai Technology)

Jul 2023 - Present

- Optimized database queries and added composite b-tree indexes on partitioned ledger tables, lowering average query execution time from 420ms to 35ms.
- Developed responsive internal operations dashboard in React and TypeScript with TanStack Query, accelerating merchant dispute resolution by 40%.
- Migrated monolithic checkout service to gRPC microservices, shrinking cloud compute costs by 32% and reducing deployment cycle times from 4 hours to 12 minutes.

PROJECTS

Peer-to-Peer File Transfer Protocol with NAT Traversal [Rust, STUN/TURN, UDP, Tokio, Cryptography]

- Developed user-space P2P file synchronization tool implementing UDP hole-punching and ICE protocol to bypass restrictive symmetric NATs.
- Utilized ChaCha20-Poly1305 authenticated encryption to guarantee confidentiality and packet integrity over public Wi-Fi networks.
- Achieved wire-speed throughput of 940 Mbps over local gigabit connections using zero-copy memory-mapped file buffers.

Distributed Log-Structured Key-Value Store [Go, Raft Consensus, gRPC, LSM-Tree, Docker]

- Implemented distributed LSM-tree storage engine in Go with write-ahead logging (WAL), SSTable compaction, and bloom filters.
- Embedded Raft consensus algorithm for distributed multi-node replication, surviving arbitrary node crashes without state divergence.
- Benchmarked performance using custom concurrent load generator, achieving 85,000 read operations/sec across a 5-node cluster.